

CSE 539 Applied Cryptography — Project Report

TLS Handshake with PAKE

Name: Diveet Kaushik

Due: May 8, 2026

1. System Architecture

1.1 Overview

In this project, I built a secure communication system between a client and a server. The server proves who it is using a certificate, and the client proves who it is using a password — but without actually sending the password over the network. This is done using a technique called PAKE (Password Authenticated Key Exchange), specifically inspired by a protocol called OPAQUE.

1.2 Parties

- **Certificate Authority (CA):** Think of this like a trusted ID card issuer. It creates a key pair and signs the server's certificate so the client knows the server is legitimate. In our project, the CA is just pre-generated files stored on disk.
- **Server:** Listens on `127.0.0.1:8443`. It has a certificate from the CA and stores each user's encrypted data after they register.
- **Client:** Only knows the CA's public key, a username, and a password. It figures out everything else during the handshake.

1.3 Components

Module	File	Responsibility
PKI & Certificates	<code>ca/ca_keygen.py</code>	Key generation, certificate signing

Digital Signature	<code>crypto/signature.py</code>	ECDSA sign, verify, load keys/certs
OPRF	<code>crypto/opr.py</code>	Oblivious PRF for password blinding
Key Derivation	<code>crypto/kdf.py</code>	HKDF extract + expand
Authenticated Encryption	<code>crypto/aes_gcm.py</code>	AES-GCM encrypt/decrypt
Server	<code>server/server.py</code>	Registration, handshake, secure channel
Client	<code>client/client.py</code>	Registration, handshake, secure channel
User Database	<code>db/users.json</code>	Stores OPRF key + encrypted envelope per user

1.4 Protocol Flow Diagram

```

CLIENT                                     SERVER
|                                         |
|--- ClientHello (username, alpha= $H(pwd)^r$ ) --->|
|                                         | [OPRF
evaluate: beta =  $alpha^k$ ]
|<-- ServerCert, beta, EphPub_S, Signature -----|
|                                         |
| [Verify CA signature on cert]                |
| [Verify server ECDSA signature]               |
| [Recover  $rw = beta^{(1/r)} = H(pwd)^k$ ]        |
| [Decrypt envelope → client keypair]           |
| [Generate ephemeral key EphPub_C]             |
|                                         |
|--- EphPub_C ----->|
|                                         |

```

```
| [Both compute: shared = ECDH(EphPriv, EphPub)] |
| [Both compute: keys = HKDF(shared, transcript)] |
|
|<-- MAC_server (key confirmation) -----|
|--- MAC_client (key confirmation) ----->|
|
|==== AES-GCM Encrypted Channel =====|
```

2. Security Intuition

2.1 Secrecy

After the handshake, all messages are encrypted using AES-GCM. This means even if someone is watching the network, they can't read what's being sent. Every message uses a new random number (nonce) so that the same message encrypted twice looks completely different.

2.2 Integrity

AES-GCM doesn't just encrypt — it also adds a tag to each message. If anyone tries to change even one bit of the message, the tag won't match and the message gets rejected. This is how we stop attackers from secretly modifying messages.

2.3 Server Authenticity

The server has an ID card (X.509 certificate) signed by the CA. During the handshake, the server also signs its messages with its private key. The client checks both of these. This way, the client can be 100% sure it's talking to the real server and not an imposter.

2.4 Client Authenticity (OPAQUE / PAKE)

This is the clever part. Usually, a client would send the password to log in. Here, we never send the password. Instead, the client and server run a mathematical trick (OPRF) where the server can verify the password is correct without ever seeing it. During registration, the client's private key is locked in a box using the password, and only someone with the right password can unlock it during login.

2.5 Forward Secrecy

Every session uses fresh temporary keys. Even if the server's main private key gets stolen someday, past conversations can't be decrypted because the temporary keys used for those sessions no longer exist.

2.6 Key Confirmation

Before starting the secure chat, both sides send each other a small proof that they derived the same keys. If the keys don't match (because someone tampered with the handshake), the connection gets rejected immediately.

3. Implementation

3.1 PKI and Certificate Generation (`ca/ca_keygen.py`)

I used Python's `cryptography` library to generate the CA's key pair and create a self-signed certificate. Then I used that CA key to sign the server's certificate. This all gets saved as `.pem` files that the server and client can read.

3.2 OPRF (`crypto/oprf.py`)

The OPRF works on elliptic curve math (P-256). The client takes the password, converts it to a curve point, and then scrambles it with a random number before sending it. The server applies its secret key to the scrambled value. The client unscrambles it to get a secret key based on the password — without the server ever seeing the actual password.

3.3 Key Derivation (`crypto/kdf.py`)

I implemented HKDF following RFC 5869. First, the raw shared secret from ECDH gets "extracted" into a proper random key. Then it gets "expanded" into multiple separate keys — one for the client, one for the server, one for the handshake, and one for the application data. Each key is tied to the transcript of the conversation so far.

3.4 Authenticated Encryption (`crypto/aes_gcm.py`)

AES-256-GCM handles all encrypted messages after the handshake. Each call picks a fresh random nonce, encrypts the message, and attaches an authentication tag. Decryption checks the tag first — if it fails, the message is dropped immediately.

3.5 Digital Signatures (`crypto/signature.py`)

The server signs its handshake messages using ECDSA with SHA-256. The client uses the CA certificate to verify the server's certificate, then uses the server's public key to verify the signature on the handshake message.

3.6 Server (`server/server.py`)

The server has two jobs:

- **Registration:** Takes the blinded password from the client, runs the OPRF, and stores the result along with the client's encrypted keypair.
- **Handshake:** Sends its signed certificate, runs the OPRF again, exchanges ephemeral keys, derives session keys using HKDF, confirms the keys with the client, and then switches to AES-GCM encrypted messaging.

3.7 Client (`client/client.py`)

The client also has two jobs:

- **Registration:** Blinds the password, sends it to the server, gets back the OPRF result, uses it to lock the client keypair in an encrypted envelope, and sends that envelope to the server.
- **Handshake:** Checks the server's certificate and signature, unlocks the envelope using the password-derived key, does the ECDH exchange, derives session keys, confirms them, and starts the encrypted chat.